

Text Editor Project - Phase 2: File Operations & Advanced Features

Overview

In this final phase, you'll transform your text editor into a fully functional program that can handle file operations and provide advanced text manipulation features.

Prerequisites

- Completed Phase 1 implementation with:
 - All basic text operations (delete, yank, paste)
 - Advanced cursor movement
 - Status line display
 - Error handling for basic operations

New Features to Implement

1. File Operations

Core Commands

- `:w filename` - Save current buffer to file
- `:q` - Quit (with warning if unsaved changes)
- `:q!` - Force quit without saving
- `:wq` - Save and quit
- `:e filename` - Open file for editing

Implementation Details

```
class FileManager {
private:
    std::string currentFileName;
    bool modified;

public:
    // Core file operations
    bool loadFile(const std::string& filename);
    bool saveFile(const std::string& filename);
    bool hasUnsavedChanges() const;

    // Helper methods
    void markAsModified();
    std::string getCurrentFileName() const;
};

class TextEditor {
```

```
private:
    FileManager fileManager;

public:
    bool handleFileCommand(const std::string& cmd);
    void updateModifiedStatus();
};
```

Status Line Enhancement

```
NORMAL | file.txt [+] | Line: 5, Col: 10
```

- Show current filename
- Show [+] if file has unsaved changes

2. Search and Replace

Commands

- `/pattern` - Search forward for pattern
- `n` - Find next occurrence
- `N` - Find previous occurrence
- `:s/old/new` - Replace first 'old' with 'new' on current line
- `:s/old/new/g` - Replace all 'old' with 'new' on current line

Implementation Details

```
class SearchEngine {
private:
    std::string lastPattern;
    size_t lastMatchLine;
    size_t lastMatchColumn;

public:
    bool search(const std::string& pattern);
    bool findNext();
    bool findPrevious();
    bool replace(const std::string& old,
                const std::string& newStr,
                bool global = false);
};
```

3. Advanced Text Operations

Commands

- **:d N** - Delete line N
- **J** - Join current line with next line
- **>>** - Indent current line
- **<<** - Unindent current line
- Number prefixes for commands:
 - **3dd** - Delete 3 lines
 - **2yy** - Yank 2 lines
 - **5j** - Move down 5 lines

Implementation

```
class TextEditor {
public:
    void joinLines();
    void indentLine(bool increase);
    void deleteLineNumber(size_t lineNum);
    void executeWithCount(int count, const std::string& cmd);
};
```

4. Command Mode Enhancement

Features

- Command history using up/down arrows
- Error messages for invalid commands
- Status messages for successful operations

Implementation

```
class CommandMode {
private:
    std::vector<std::string> commandHistory;
    size_t historyIndex;
    std::string statusMessage;

public:
    void addToHistory(const std::string& cmd);
    std::string getPreviousCommand();
    std::string getNextCommand();
    void setStatus(const std::string& msg);
};
```

Error Handling Requirements

1. File Operations

- File not found
- Permission denied
- Disk full
- File already exists

```
void handleFileError(const std::string& operation,
                    const std::string& filename,
                    int errorCode);
```

2. Search and Replace

- Pattern not found
- Invalid pattern
- Empty search string

```
void handleSearchError(const std::string& pattern,
                      SearchErrorType error);
```

3. Command Mode

- Invalid command syntax
- Non-existent line numbers
- Invalid operation

```
void handleCommandError(const std::string& cmd,
                       CommandErrorType error);
```

Testing Requirements

1. File Operation Tests

```
void testFileOperations() {
    // Test file loading
    assert(editor.loadFile("test.txt"));

    // Test file saving
    editor.insertText("Test content");
    assert(editor.saveFile("output.txt"));

    // Test unsaved changes
    assert(editor.hasUnsavedChanges());
}
```

2. Search Tests

```
void testSearch() {  
    // Test pattern finding  
    editor.loadFile("test.txt");  
    assert(editor.search("pattern"));  
    assert(editor.findNext());  
  
    // Test replacement  
    assert(editor.replace("old", "new"));  
}
```

3. Command Tests

```
void testCommands() {  
    // Test line operations  
    editor.executeCommand(":d 5");  
    editor.executeCommand("3dd");  
  
    // Test indentation  
    editor.executeCommand(">>");  
}
```

Evaluation Criteria

1. File operations reliability
2. Search and replace accuracy
3. Command parsing robustness
4. Error handling completeness
5. Code organization

Submission Requirements

1. Complete source code
2. Sample text files for testing

Remember: Prioritize reliability and error handling over adding new features. A stable, well-tested implementation is more important than a feature-rich but unstable one.